SOFTWARE FUNCTIONALITY

Group members

Janan Alsaedi - 444005198
Hadeel Kufiah - 444007458
Amjaad Hawsawi - 444003283
Weaam Rashed - 444015412
Norah Alkabkabi - 444003676

OVERVIEW

- Definition
- Types of Functionality
- Evaluating Software Functionality
- Importance of Software Functionality
- Interaction Between Software Functions
- Software Functionality Latest Trends

DEFINITION

Software functionality refers to the specific capabilities and features that a software application provides to meet the needs and requirements of its users. It encompasses all the tasks and operations that a software system is designed to perform, from basic processes to more advanced interactions with the user or other systems.

In technical terms, software functionality represents the intended behavior of the software, driven by its design and the problem it seeks to solve. It outlines how the software behaves when interacting with users, hardware, and external systems, and what outputs or results it generates.

"Software Engineering: A Practitioner's Approach" by Roger S. Pressman: This textbook provides comprehensive coverage of software development, including the role of functionality in meeting user needs.

TYPES OF FUNCTIONALITY

CORE FUNCTIONALITY

These are the essential features that define the software's primary purpose. They are the fundamental functions needed for the software to operate correctly.

Example:
Word Processor: Text editing

Purpose: Without core functionality, the software cannot fulfill its basic role

ADDITIONAL FUNCTIONALITY

Features that enhance the user experience but are not essential for the software's primary operation.

Example:
Photo Editing Software: Filters, effects (can work without these features but greatly enhance the user experience)

Purpose: increases usability, but removing these features doesn't affect core functionality.

"Software Engineering: A Practitioner's Approach" by Roger S. Pressman

TYPES OF FUNCTIONALITY

USER INTERFACE (UI) FUNCTIONALITY

facilitate user interaction with the software. The design and responsiveness of the interface significantly impact user satisfaction and ease of use.

Example: Responsive Design, Adapts the UI to different screen sizes and devices.

Purpose: A well-designed user interface improves user experience, accessibility, and efficiency in interacting with the software.

DATA MANAGEMENT FUNCTIONALITY

How the software manages, processes, stores, and retrieves data. Effective data management ensures that users can work with large volumes of information reliably.

Example: CRM Systems: Customer data entry, storage in databases, data retrieval via search.

Purpose: Proper data management ensures data integrity, accessibility.

"Software Engineering: A Practitioner's Approach" by Roger S. Pressman

TYPES OF FUNCTIONALITY

INTEGRATION FUNCTIONALITY

The ability of the software to connect and interact with other systems or external applications, enhancing its capabilities.

Example: Social Media Apps: Integration with external platforms (e.g., login with Facebook,)

Purpose: Integration enhances the software's reach and versatility by enabling it to communicate and share data with other services and platforms.

SECURITY FUNCTIONALITY

Features designed to safeguard the software and its data from unauthorized access, malicious attacks, and data breaches.

Example: Authentication, User login with passwords, two-factor authentication.

Purpose: Security functionality ensures that users' data is protected, builds trust, and helps comply with regulations like GDPR.

"Software Engineering: A Practitioner's Approach" by Roger S. Pressman

EVALUATING SOFTWARE FUNCTIONALITY

What is Evaluating Software Functionality?

- Definition: Assessing how well software performs its intended tasks and meets user needs.
- Importance: Ensures quality, performance, and user satisfaction.
- Key Areas: Core functions, additional features, UI, data management, function interaction.

Pressman, R. S. (2021). Software Engineering: A Practitioner's Approach.

Sommerville, I. (2020). Software Engineering.

EVALUATING SOFTWARE FUNCTIONALITY

Evaluation Process

- Define Requirements: Specify what the software must do.
- Research Solutions: Shortlist based on features and reviews.
- Test Functionality: Check core/additional features, UI, data, and interactions.
- Performance Testing: Measure speed, scalability, and stability.
- User Feedback: Gather input to confirm user needs are met.

Pressman, R. S. (2021). Software Engineering: A Practitioner's Approach.

Sommerville, I. (2020). Software Engineering.

THE IMPORTANCE OF SOFTWARE FUNCTIONALITY

The main goal of functionality in software engineering is to provide users with a seamless and intuitive software interface that allows them to accomplish their tasks easily.
The role of functionality in software development is paramount. It sets the basis for the entire software engineering process.

-Institute of Data, “The Importance of Functionality in Software Engineering,” Feb. 23, 2024.

THE IMPORTANCE OF SOFTWARE FUNCTIONS

The functionality of a software application directly impacts user satisfaction and engagement. When users can easily navigate the software and accomplish their tasks efficiently, their overall experience improves.

-Institute of Data, “The Importance of Functionality in Software Engineering,” Feb. 23, 2024.

THE IMPORTANCE OF SOFTWARE FUNCTIONALITY

Functionality in software engineering includes straightforward navigation, clear instructions, and logical workflows.
These elements contribute to an improved user experience, resulting in higher satisfaction.
Furthermore,functionality influences user engagement with the software.

-Institute of Data, “The Importance of Functionality in Software Engineering,” Feb. 23, 2024.

INTERACTION BETWEEN SOFTWARE FUNCTIONS

Software functions interact to perform tasks smoothly. Good interaction enhances user experience, while poor interaction causes delays and errors.

HOW SOFTWARE FUNCTIONS INTERACT

Every software system is made up of different functions that must work together to provide a smooth user experience.

"Research on Usability and User Experience of UI Design."

EXAMPLES OF FUNCTION INTERACTION

E-commerce Apps

- **Search function** retrieves product data from database.
- **Cart function** interacts with inventory to check stock.
- **Payment function** verifies card details and updates order status.

"Research on Usability and User Experience of UI Design."

IMPACT ON USER EXPERIENCE

Good function interaction leads to:

- **Smooth navigation**
- **Faster response times**
- **Better reliability**

Poor function interaction leads to:

- **Slow performance**
- **Errors and crashes**
- **Low engagement**

“The Impact of User Interface Design on User Engagement.”

SOFTWARE FUNCTIONALITIES

LATEST TRENDS

1 LOW-CODE / NO-CODE

Low-Code Platforms: Require some coding for customization, making them ideal for developers who want to accelerate development while maintaining flexibility.

No-Code Platforms: Designed for non-technical users, allowing complete application creation without writing any code.

Top 13 Software Engineering Trends to Watch for in 2025-clickup artical

1 LOW-CODE / NO-CODE

Advantages:
1- Lowers barriers to application development.
2- Reduces the need for specialized coding skills, cutting costs on hiring developers.
3- Speeds up development cycles and enhances productivity.

Tools:

- Microsoft PowerApps
- OutSystems
- Mendix

Top 15 Low Code No Code Platforms in 2025-Dev artical

2 EDGE COMPUTING FRAMEWORK

- Edge computing is a distributed computing framework that allows IoT devices to process and act on data closer to the data source.

Advantages for Businesses:

1- Improves response times for remote devices.

2- Provides richer, more timely insights from device data.

3- Reduces latency and bandwidth usage, optimizing network performance.

Tools:

- Azure IoT Edge
- Azure Stack Edge
- Azure Sphere

What is edge computing? - Microsoft Azura article

3 DEVSECOPS INTEGRATION

What is DevSecOps?
DevSecOps integrates security testing at every stage of the software development lifecycle. It fosters collaboration between developers, security teams, and operations, ensuring software is both efficient and secure.

What is DevSecOps?-AWS article

3 DEVSECOPS INTEGRATION

Advantages:

1- Detects vulnerabilities early in development.
2- Reduces time to market while ensuring security.
3- Ensures compliance with regulatory standards.
4- Builds a security-aware culture within teams.

Tools:

- Static Application Security Testing (SAST)
- Software Composition Analysis (SCA)
- Interactive Application Security Testing (IAST)

4 MICROSERVICES FOR COMPLEX TASKS

Microservices architecture involves breaking applications into smaller, independent services, which can be developed, deployed, and scaled separately. This enhances flexibility, scalability, and maintainability.

Microservices architecture and design: A complete overview-vFunction article

4 MICROSERVICES FOR COMPLEX TASKS

Advantages:

1- Enables platforms to break down functionalities for better modularity.

2- Enhances reliability—critical services remain operational even if other components fail.

3- Allows different teams to update and scale services independently.

Tools:

- Docker
- Kubernetes
- Istio
- AWS Lambda

THANK YOU
FOR YOUR TIME.